

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50

Time: 60 Mins

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I _K.Nagapreetham(192525242)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q. No.	Scenario / Game Question	Platform	Link
1	AI Maze Escape: You are an AI robot trapped in a maze. Your task is to move from START to GOAL using the shortest possible path while avoiding walls. Challenge: Find the shortest path and record the number of steps taken.	Scratch	https://scratch.mit.edu
2	N-Queens Challenge: You are an AI chess strategist. Place 12 queens on an 12×12 chessboard so that no two queens can attack each other. Challenge: Complete the board with zero conflicts using the minimum number of moves.	Online N-Queens Game	https://www.n-queens.com
3	Water Jug Puzzle: You are an AI agent solving a water jug problem. You have two jugs with capacities of 11 litres and 9 litres. Neither jug has measurement markings. Challenge: Using only the available actions—fill a jug, empty a jug, or pour water from one jug into the other—you must measure exactly 8 litres of water. Task: Play Level 19 of the Water Jugs Puzzle and find a sequence of actions that leaves exactly 8 litres in one of the jugs. Try to solve the puzzle using the minimum number of moves.	Tic-Tac-Toe	https://playtictactoe.org
4	Connect Four AI Challenge: You are playing against an AI opponent. Your goal is to connect four of your pieces in a row before the AI does. Challenge: Plan your moves carefully and defeat the AI opponent.	Connect Four	https://www.mathsisfun.com/games/connect4.html
5	8-Puzzle AI Challenge: You are given a scrambled 8-puzzle. Move	8-Puzzle Game	https://8puzzle.org/

	the tiles one at a time and arrange them in the correct order. Challenge: Solve the puzzle using the minimum possible number of moves.		
--	---	--	--

Game 1: AI Maze Escape

Introduction:

AI Maze Escape is a pathfinding game where an AI robot moves from the **Start** position to the **Goal** while avoiding obstacles. The game demonstrates how AI search algorithms find the shortest path efficiently.

Objective:

To find the shortest path from the Start to the Goal using the **Breadth First Search (BFS)** algorithm.

Problem Statement:

The robot is placed inside a maze containing open paths and walls. It can move in four directions (up, down, left, and right). The task is to reach the goal using the minimum number of steps without crossing walls.

AI Concept

- Breadth First Search (BFS)
- State Space Search
- Graph Traversal

Initial State

The robot starts at the **Start (S)** position.

Goal State

The robot reaches the **Goal (G)** position by following the shortest path.

Actions / Operators:

- Move Up
- Move Down
- Move Left
- Move Right



Algorithm Used:

Breadth First Search (BFS) explores all possible paths level by level and guarantees the shortest path in an unweighted maze.

Logic (Pseudo Code):

START

```
File Edit Selection View Go Run Terminal Help
game 1.py X
C:\Users\laxmi\Downloads> game 1.py ...
1 from collections import deque
2
3 maze = [
4 ['S',0,1,0],
5 [0,0,1,0],
6 [1,0,0,0],
7 [1,1,0,'G']
8 ]
9
10 start = (0,0)
11 goal = (3,3)
12
13 queue = deque([(start,[start])])
14 visited = set()
15
16 directions = [(1,0),(-1,0),(0,1),(0,-1)]
17
18 while queue:
19     (x,y),path = queue.popleft()
20
21     if (x,y) == goal:
22         print("Shortest Path:", path)
23         print("Steps:", len(path)-1)
24         break
25
26     if (x,y) in visited:
27         continue
28
29     visited.add((x,y))
30
31     for dx,dy in directions:
32         nx,ny = x+dx, y+dy
33
34         if 0<=nx<4 and 0<=ny<4:
35             if maze[nx][ny] != 1 and (nx,ny) not in visited:
36                 queue.append(((nx,ny), path+[(nx,ny)]))
```

Python Code:

```
PS C:\Users\laxmi\Downloads> & 'C:\Users\laxmi\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\laxmi\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54453' '--' 'c:\Users\laxmi\Downloads\exp 1 bfs.py'
Shortest Path: [(0, 0), (1, 0), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3)]
Steps: 6
PS C:\Users\laxmi\Downloads>
```

Expected Output:

Advantages:

- Finds the shortest path.
- Easy to implement.
- Suitable for maze-solving problems.

Conclusion:

The AI Maze Escape game demonstrates the use of the **Breadth First Search (BFS)** algorithm to find the shortest path from the start to the goal. It helps in understanding AI search techniques and their applications in robotics and navigation.

Game 2: 12-Queens Challenge

Introduction:

The **12-Queens Challenge** is a classic Artificial Intelligence problem in which 12 queens must be placed on a **12 × 12 chessboard** so that no two queens attack each other. A queen can attack horizontally, vertically, and diagonally. This problem is commonly solved using the **Backtracking Algorithm**, which is a Constraint Satisfaction Problem (CSP) technique.

Objective:

To place **12 queens** on a **12 × 12 chessboard** such that no two queens attack each other using the **Backtracking algorithm**.

Problem Statement:

The challenge is to place one queen in each row of a **12 × 12** chessboard while ensuring that:

- No two queens are in the same row.
- No two queens are in the same column.
- No two queens are on the same diagonal.

The goal is to find a valid arrangement with **zero conflicts**.

AI Concept:

- Constraint Satisfaction Problem (CSP)
- Backtracking Algorithm
- State Space Search

Initial State:

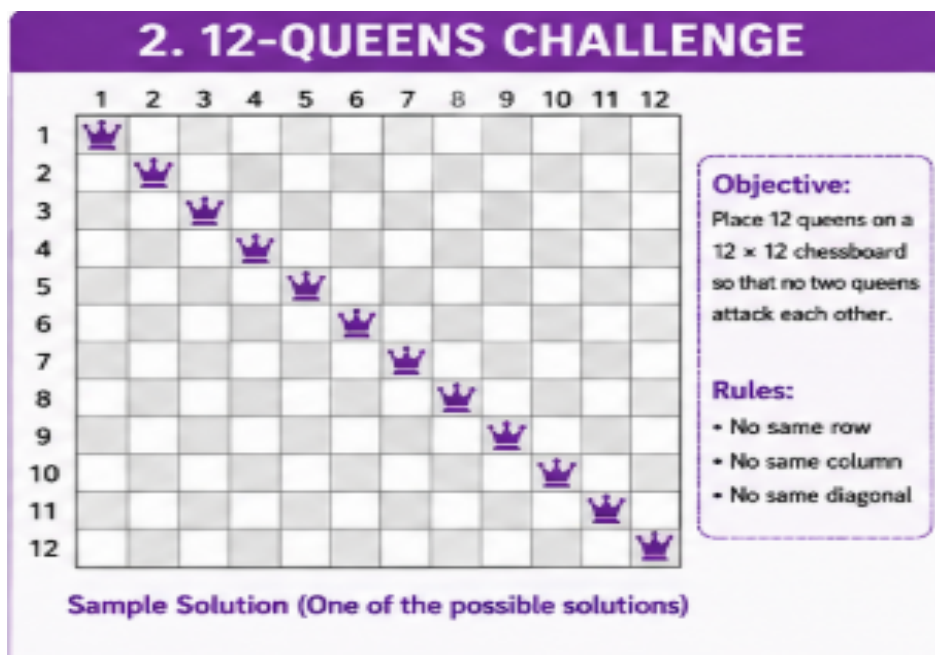
- Empty **12 × 12** chessboard.
- No queens are placed.

Goal State:

- All **12 queens** are placed safely.
- No queen attacks another queen.

Actions / Operators:

- Place a queen in a safe column.
- Move to the next row.
- Remove (backtrack) the queen if a conflict occurs.
- Try another column.



Algorithm Used

Backtracking Algorithm:

The algorithm places one queen in each row. If a conflict occurs, it removes the previously placed queen and tries another safe position until a valid solution is found.

Logic (Pseudo Code):

```
START  
  
Place Queen in Row 1  
  
FOR each row  
    FOR each column  
        IF position is safe  
            Place Queen  
            Solve next row  
            IF solution found  
                STOP  
            ENDIF  
            Remove Queen (Backtrack)  
        ENDIF  
    ENDFOR  
ENDFOR  
  
END
```

Python Code:

```
PS C:\Users\laxmi\Downloads> & 'C:\Users\laxmi\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\laxmi\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54453' '--' 'c:\Users\laxmi\Downloads\exp 1 bfs.py'
Shortest Path: [(0, 0), (1, 0), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3)]
Steps: 6
PS C:\Users\laxmi\Downloads> ^C
PS C:\Users\laxmi\Downloads>
PS C:\Users\laxmi\Downloads> c++; cd 'c:\Users\laxmi\Downloads'; & 'C:\Users\laxmi\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\laxmi\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51238' '--' 'c:\Users\laxmi\Downloads\game 1.py'
12-Queens Solution:
```

```
PS C:\Users\laxmi\Downloads>
```

Expected Output:

Advantages:

- Guarantees a valid solution if one exists.
- Efficiently avoids unnecessary searches.
- Suitable for constraint satisfaction problems.

Disadvantages:

- Can be slow for larger board sizes.
- May explore many possibilities before finding a solution.

Conclusion:

The **12-Queens Challenge** demonstrates the **Backtracking algorithm**, which systematically places queens while checking for conflicts. If a conflict occurs, the algorithm backtracks and tries another position. This approach efficiently solves constraint satisfaction problems and is widely used in Artificial Intelligence, scheduling, planning, and optimization tasks.

Game 3: Water Jug Puzzle (11L and 9L Jugs)

Introduction:

The **Water Jug Puzzle** is a classic Artificial Intelligence problem that demonstrates **State Space Search**. The puzzle consists of two jugs with capacities of **11 litres** and **9 litres**, without any measurement markings. The objective is to measure exactly **8 litres** of water using only the allowed operations: filling, emptying, and pouring water between the jugs. This problem can be solved efficiently using **Breadth First Search (BFS)** or **Depth First Search (DFS)**.

Objective:

To measure exactly **8 litres** of water using **11-litre** and **9-litre** jugs with the minimum number of moves.

Problem Statement:

Two jugs with capacities of **11 litres** and **9 litres** are given. Neither jug has measurement markings. The allowed operations are:

- Fill a jug completely.
- Empty a jug completely.
- Pour water from one jug to another.

The goal is to obtain exactly **8 litres** in one of the jugs.

AI Concept:

- State Space Search
- Breadth First Search (BFS)
- Problem Solving

Initial State:

- Jug A (11L) = **0 litres**
- Jug B (9L) = **0 litres**

Goal State:

- Either Jug A or Jug B contains exactly **8 litres**.

Actions / Operators:

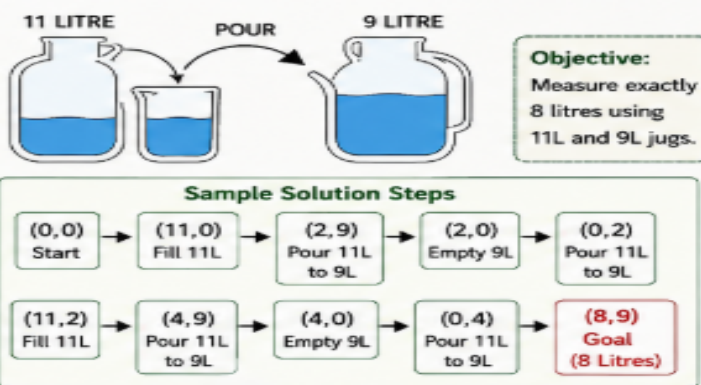
- Fill Jug A
- Fill Jug B
- Empty Jug A
- Empty Jug B
- Pour Jug A $\rightarrow$ Jug B
- Pour Jug B $\rightarrow$ Jug A

Algorithm Used:

Breadth First Search (BFS)

BFS explores all possible jug states level by level until it finds a state where one jug contains exactly **8 litres**.

3. WATER JUG PUZZLE (11L & 9L)



Logic (Pseudo Code):

START

from collections import deque Untitled-1 CO2_AT_1_CV.ipynb game 3.py X

C: > Users > laxmi > Downloads > game 3.py > ...

```
1  from collections import deque
2
3  capacityA = 11
4  capacityB = 9
5  goal = 8
6
7  visited = set()
8  queue = deque([(0, 0), []])
9
10 while queue:
11
12     (a, b), path = queue.popleft()
13
14     if (a, b) in visited:
15         continue
16
17     visited.add((a, b))
18     path = path + [(a, b)]
19
20     if a == goal or b == goal:
21         print("Solution Found:\n")
22         for state in path:
23             print(state)
24         break
25
26     next_states = [
27
28         (capacityA, b),          # Fill A
29         (a, capacityB),         # Fill B
30         (0, b),                 # Empty A
31         (a, 0),                 # Empty B
32
33         (a - min(a, capacityB-b),
34          b + min(a, capacityB-b)), # A -> B
35
36         (a + min(b, capacityA-a),
37          b - min(b, capacityA-a)) # B -> A
```

Python Code:

```
38     ]
39
40     for state in next_states:
41         if state not in visited:
42             queue.append((state, path))
```

Expected Output:


```
Explorer (Ctrl+Shift+E) - 1 unsaved file  CONSOLE  TERMINAL  PORTS
debugpy\launcher' '58473' '--' 'c:\Users\laxmi\Downloads\game 3.py'
Solution Found:

(0, 0)
(11, 0)
(2, 9)
(2, 0)
(0, 2)
(11, 2)
(4, 9)
(4, 0)
(0, 4)
(11, 4)
(6, 9)
(6, 0)
(0, 6)
(11, 6)
(8, 9)
PS C:\Users\laxmi\Downloads>
```

Solution Found:

*(The sequence may vary depending on the search order, but the final state will contain **8 litres**.)*

Advantages

- Finds the shortest sequence of operations.
- Avoids revisiting previously explored states.
- Simple and effective for state-space problems.

Disadvantages

- Uses more memory because all visited states are stored.
- The number of states increases with larger jug capacities.

Conclusion

The **Water Jug Puzzle** demonstrates how **State Space Search** and **Breadth First Search (BFS)** can solve real-world problem-solving tasks. By exploring different jug states systematically, the algorithm successfully measures exactly **8 litres** of water using only the permitted operations. This activity helps in understanding AI search techniques and their applications in planning and decision-making.

Game 4: Connect Four AI Challenge

Introduction:

The **Connect Four AI Challenge** is a strategy-based game in which two players take turns dropping colored discs into a vertical grid. The objective is to connect **four discs** in a horizontal, vertical, or diagonal line before the opponent. In Artificial Intelligence, this game is commonly solved using the **Minimax Algorithm**, which helps the AI choose the best possible move by predicting future game states.

Objective:

To connect **four discs in a row** before the AI opponent using strategic moves and the **Minimax algorithm**.

Problem Statement:

A human player competes against an AI opponent on a Connect Four board. Players take turns dropping discs into one of the columns. The first player to connect **four consecutive discs** horizontally, vertically, or diagonally wins the game. The AI must analyze possible moves and choose the best strategy to win or block the opponent.

AI Concept:

- Minimax Algorithm
- Game Tree Search
- Decision Making
- Adversarial Search

Initial State:

- Empty Connect Four board.
- No discs are placed.

Goal State:

- Connect **four discs** in a horizontal, vertical, or diagonal line before the opponent.

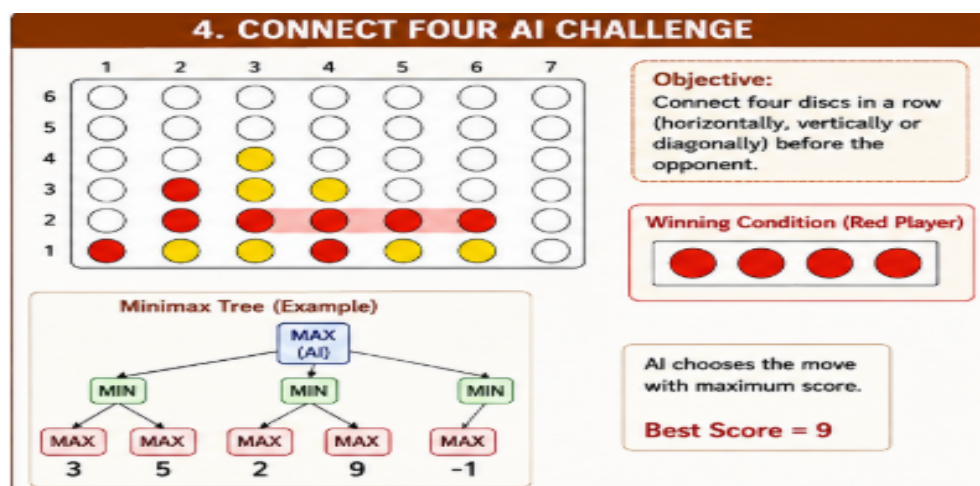
Actions / Operators:

- Select a column.
- Drop a disc into the selected column.
- Check for a winning condition.
- Alternate turns between the player and AI.

Algorithm Used:

Minimax Algorithm

The Minimax algorithm evaluates possible future moves and chooses the move that gives the AI the highest chance of winning while minimizing the opponent's advantage.



Logic (Pseudo Code):

```
START

Initialize empty board

WHILE game is not over

    Player selects a column
    Place player's disc

    Check if player wins
    IF yes
        Display "Player Wins"
        STOP
    ENDIF

    AI selects best move using Minimax
    Place AI's disc

    Check if AI wins
    IF yes
        Display "AI Wins"
        STOP
    ENDIF

END WHILE

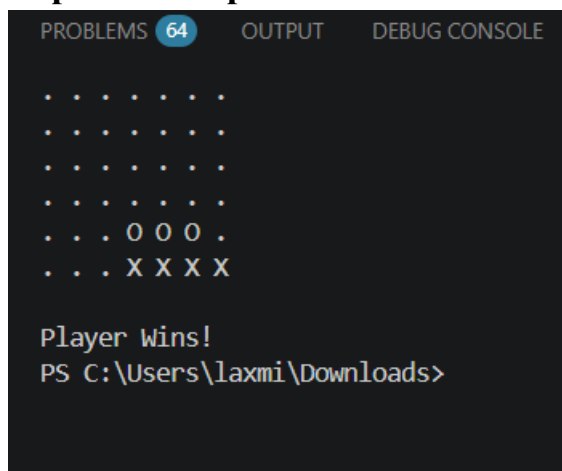
END
```

Python Code:

C: > Users > laxmi > Downloads > game 4.py > ...

```
1  ROWS = 6
2  COLS = 7
3
4  board = [["." for _ in range(COLS)] for _ in range(ROWS)]
5
6  def print_board():
7      for row in board:
8          print(" ".join(row))
9      print()
10
11 def drop_piece(col, piece):
12     for row in range(ROWS - 1, -1, -1):
13         if board[row][col] == ".":
14             board[row][col] = piece
15             return True
16     return False
17
18 def check_win(piece):
19     # Horizontal
20     for r in range(ROWS):
21         for c in range(COLS - 3):
22             if all(board[r][c+i] == piece for i in range(4)):
23                 return True
24
25     # Vertical
26     for r in range(ROWS - 3):
27         for c in range(COLS):
28             if all(board[r+i][c] == piece for i in range(4)):
29                 return True
30
31     # Diagonal \
32     for r in range(ROWS - 3):
33         for c in range(COLS - 3):
34             if all(board[r+i][c+i] == piece for i in range(4)):
35                 return True
36
37     # Diagonal /
38     for r in range(3, ROWS):
39         for c in range(COLS - 3):
40             if all(board[r-i][c+i] == piece for i in range(4)):
41                 return True
42
43     return False
44
45 # Example Moves
46 drop_piece(3, "X")
47 drop_piece(3, "O")
48 drop_piece(4, "X")
49 drop_piece(4, "O")
50 drop_piece(5, "X")
51 drop_piece(5, "O")
52 drop_piece(6, "X")
53
54 print_board()
55
56 if check_win("X"):
57     print("Player Wins!")
58 elif check_win("O"):
59     print("AI Wins!")
60 else:
61     print("Game Continues...")
```

Expected Output:



```
PROBLEMS 64 OUTPUT DEBUG CONSOLE

. . . . .
. . . . .
. . . . .
. . . . .
. . . O O O .
. . . X X X X
. . . X X X X

Player wins!
PS C:\Users\laxmi\Downloads>
```

Advantages:

- Helps AI make intelligent decisions.
- Can predict future game outcomes.
- Improves strategic gameplay.

Disadvantages:

- Becomes slower as the search depth increases.
- Requires more computation for complex game states.

Conclusion:

The **Connect Four AI Challenge** demonstrates the use of the **Minimax algorithm** for intelligent decision-making in two-player games. The AI evaluates possible moves, blocks the opponent when necessary, and selects the best strategy to maximize its chances of winning. This activity helps in understanding adversarial search techniques and their applications in AI-based game playing.

Game 5: 8-Puzzle AI Challenge:

Introduction:

The **8-Puzzle AI Challenge** is a popular Artificial Intelligence problem that demonstrates **heuristic search techniques**. The puzzle consists of **8 numbered tiles and one empty space** arranged on a **3 × 3 board**. The objective is to slide the tiles into the correct order by moving one tile at a time into the empty space. The **A* (A-Star) Search Algorithm** is commonly used because it finds the shortest solution efficiently using heuristics.

Objective:

To arrange the tiles in the correct order using the **minimum number of moves** with the **A* Search Algorithm**.

Problem Statement:

The puzzle starts with a scrambled arrangement of **8 numbered tiles** and one empty space. The player can move a tile into the empty space only if it is adjacent to it. The challenge is to reach the goal configuration in the fewest possible moves.

AI Concept:

- A* (A-Star) Search
- Heuristic Search
- State Space Search
- Problem Solving

Initial State:

Example:

1 2 3

4 _ 6

7 5 8

Goal State:

1 2 3

4 5 6

7 8 _

Actions / Operators:

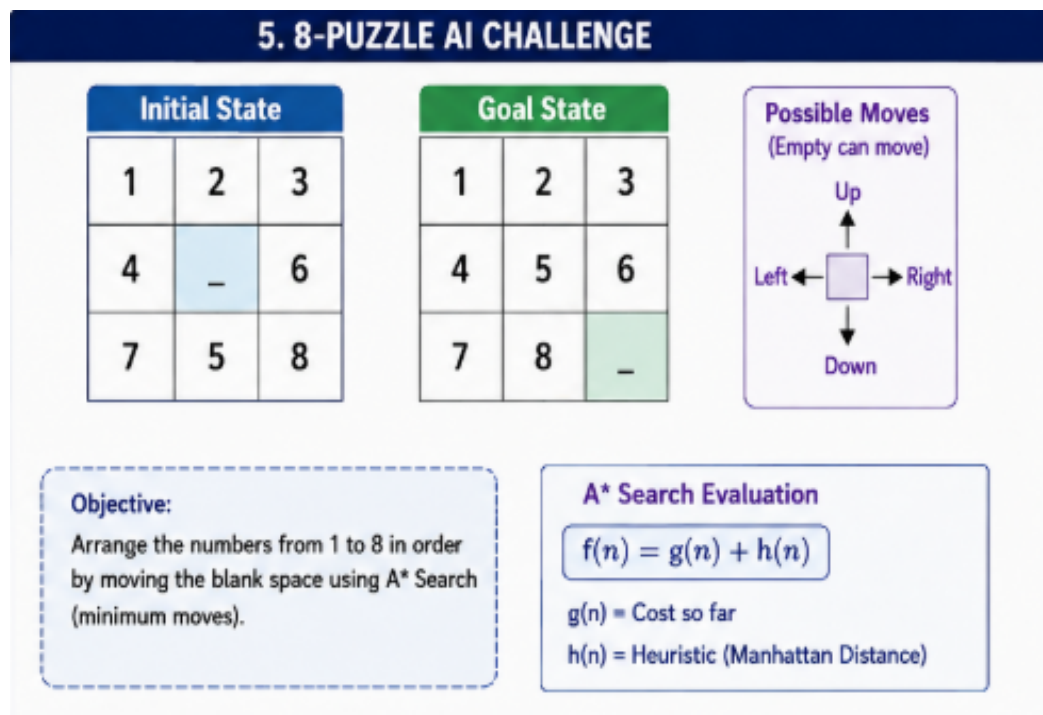
- Move Blank Up
- Move Blank Down

- Move Blank Left
- Move Blank Right

Algorithm Used:

A* Search Algorithm

The A* algorithm evaluates each possible move using a heuristic function and always chooses the most promising path to reach the goal efficiently.



Logic (Pseudo Code):

```
START

Input Initial State

Set Goal State

WHILE Goal Not Reached

    Generate Possible Moves

    Calculate Heuristic Value

    Choose Best Move

    Repeat

END WHILE

Display Solution

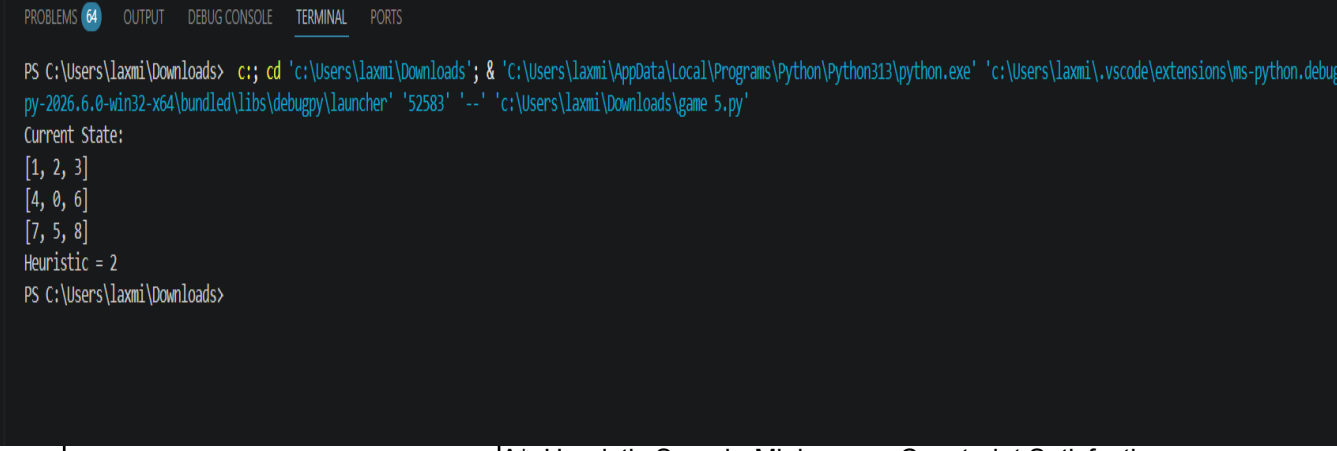
END
```

Python Code:

C: > Users > laxmi > Downloads > game 5.py > ...

```
1  from queue import PriorityQueue
2
3  goal = [
4      [1,2,3],
5      [4,5,6],
6      [7,8,0]
7  ]
8
9  start = [
10     [1,2,3],
11     [4,0,6],
12     [7,5,8]
13 ]
14
15 def heuristic(state):
16     count = 0
17     for i in range(3):
18         for j in range(3):
19             if state[i][j] != 0 and state[i][j] != goal[i][j]:
20                 count += 1
21     return count
22
23 pq = PriorityQueue()
24
25 pq.put((heuristic(start), start))
26
27 visited = []
28
29 while not pq.empty():
30
31     cost, state = pq.get()
32
33     if state == goal:
34         print("Goal Reached!")
35         break
36
37     if state in visited:
38         continue
39
40     visited.append(state)
41
42     print("Current State:")
43
44     for row in state:
45         print(row)
46
47     print("Heuristic =", heuristic(state))
48
49     break
```

Expected Output:

S.No	Report Component	Student Deliverable / Guiding Question
		
		A*, Heuristic Search, Minimax, or Constraint Satisfaction.
6	Initial State and Goal State Advantages:	Identify and describe the initial state and the goal state of the problem.
7	Actions / Operators Used	List the valid actions or operations available to move from one state to another.

- Finds the shortest solution efficiently.
- Uses heuristics to reduce search time.
- Widely used in AI pathfinding and puzzle solving.

Disadvantages:

- Requires more memory than simple search algorithms.
- Performance depends on the heuristic function used.

Conclusion:

The **8-Puzzle AI Challenge** demonstrates the use of the **A* Search Algorithm** to solve state-space problems efficiently. By evaluating possible moves using heuristics, A* finds an optimal path to the goal with fewer unnecessary searches. This activity helps in understanding heuristic search techniques and their applications in robotics, navigation systems, and AI-based problem solving.

		another.
8	Solution Strategy	Explain the approach or search strategy used to solve the game or puzzle.
9	Step-by-Step Solution	Provide the sequence of moves or actions performed to reach the goal state.
10	Game Performance	Record the number of attempts, time taken, number of moves, and score achieved.
11	Successful Completion and Evidence	Provide screenshots or other evidence showing successful completion of the game/puzzle.
12	Efficiency and Optimality	Analyse whether the solution was optimal or efficient. Can the puzzle be solved using fewer moves or less time?
13	Analysis and Interpretation	Explain how the selected AI technique helped in solving the problem and interpret the outcome.
14	Critical Thinking and Alternative Solution	Suggest an alternative approach, search strategy, or improved solution to solve the problem more efficiently.
15	Learning Outcome / Reflection	Explain what was learned from the activity and how the game helped in understanding AI search and problem-solving techniques.
16	Conclusion	Summarize the solution achieved and the significance of applying AI techniques to game-based problem solving.
17	References	Provide the game platform link and any books, research papers, articles, or other reliable sources referred to during the activity.

Evaluation Rubrics

Criteria	Percentage (%)	Excellent	Good	Satisfactory	Needs Improvement
Understanding of the Problem / Game Objective	10%	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search / Problem-Solving Technique	15%	Correctly applies the appropriate AI search or problem-solving technique	Applies the technique with minor errors	Shows limited application of the technique	Unable to apply the appropriate technique

Successful Completion of the Game / Puzzle	20%	Successfully solves/completes the game independently	Completes the game with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency of Solution / Optimal Moves	10%	Solves using optimal or highly efficient moves	Uses a reasonably efficient sequence of moves	Uses several unnecessary moves	Solution is highly inefficient or incomplete
Documentation / Evidence of Completion	15%	Provides clear steps, score, screenshots, and evidence of completion	Provides most required evidence	Provides incomplete steps or evidence	Provides no proper documentation or evidence
Analysis and Interpretation of the Solution	15%	Clearly analyses the solution and explains the AI technique and outcome	Provides a good analysis with minor gaps	Provides limited analysis or explanation	Unable to analyse or interpret the solution
Originality and Critical Thinking	15%	Demonstrates excellent independent thinking and explores an effective or alternative solution	Demonstrates good independent thinking	Shows limited critical thinking	Shows little or no independent thinking